

```

# =====
# TASK 3
# Model Validation, Overfitting Control &
# Hyperparameter Tuning
# =====

# Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import GridSearchCV

from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Load Dataset
housing = fetch_california_housing()

X = pd.DataFrame(housing.data, columns=housing.feature_names)
y = housing.target

print("Dataset Shape:", X.shape)
print(X.head())

# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)

# Decision Tree Model
model = DecisionTreeRegressor(random_state=42)

model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Evaluation
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("RMSE =", rmse)
print("R2 Score =", r2)

# Cross Validation
scores = cross_val_score(
    model,
    X,
    y,
    cv=5,
    scoring="r2"
)

print("Cross Validation Scores")
print(scores)

print("Average CV Score =", scores.mean())

```

```

Dataset Shape: (20640, 8)
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0   8.3252    41.0    6.984127   1.023810     322.0    2.555556    37.88
1   8.3014    21.0    6.238137   0.971880    2401.0    2.109842    37.86
2   7.2574    52.0    8.288136   1.073446     496.0    2.802260    37.85
3   5.6431    52.0    5.817352   1.073059     558.0    2.547945    37.85
4   3.8462    52.0    6.281853   1.081081     565.0    2.181467    37.85

```

```
Longitude
0    -122.23
1    -122.22
2    -122.24
3    -122.25
4    -122.25
Training Data: (16512, 8)
Testing Data: (4128, 8)
RMSE = 0.7037294974840077
R2 Score = 0.622075845135081
Cross Validation Scores
[0.27093461 0.41372445 0.43912441 0.23566991 0.41875969]
Average CV Score = 0.355642615410327
```

✓ New Section

```
# Hyperparameter Tuning using GridSearchCV

params = {
    'max_depth': [3, 5, 7, 10, None],
    'min_samples_split': [2, 5, 10]
}

grid = GridSearchCV(
    DecisionTreeRegressor(random_state=42),
    param_grid=params,
    cv=5,
    scoring='r2'
)

grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)
print("Best Cross Validation Score:", grid.best_score_)

# Best Model
best_model = grid.best_estimator_

# Prediction
best_pred = best_model.predict(X_test)

# Evaluation
rmse = np.sqrt(mean_squared_error(y_test, best_pred))
r2 = r2_score(y_test, best_pred)

print("Final RMSE:", rmse)
print("Final R2 Score:", r2)

# Actual vs Predicted Graph
plt.figure(figsize=(6,6))
plt.scatter(y_test, best_pred)
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Actual vs Predicted")
plt.grid(True)
plt.show()

# Feature Importance Graph
importance = best_model.feature_importances_

plt.figure(figsize=(8,5))
plt.bar(X.columns, importance)
plt.xticks(rotation=45)
plt.title("Feature Importance")
plt.show()
```

Best Parameters: {'max_depth': 10, 'min_samples_split': 10}
Best Cross Validation Score: 0.6959937053519777
Final RMSE: 0.6456884926536758
Final R2 Score: 0.6818446481456171

